Data Structures and Algorithm

Xiaoqing Zheng
zhengxq@fudan.edu.cn







Deep learning and big data

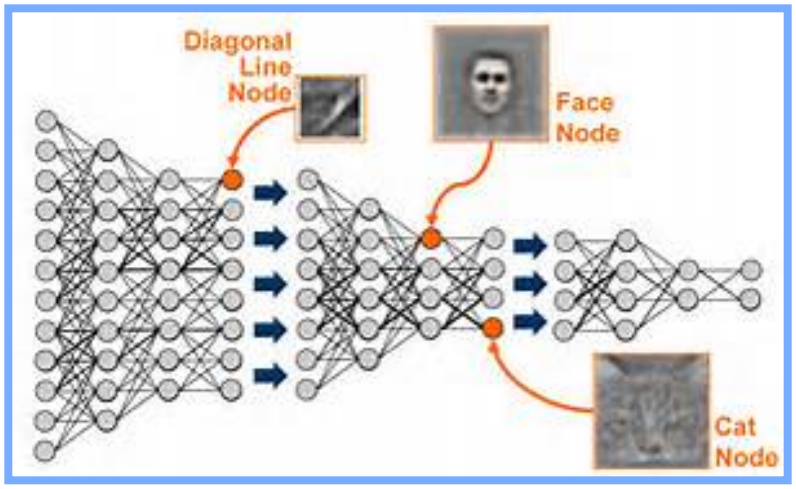






Deep learning

Big Data





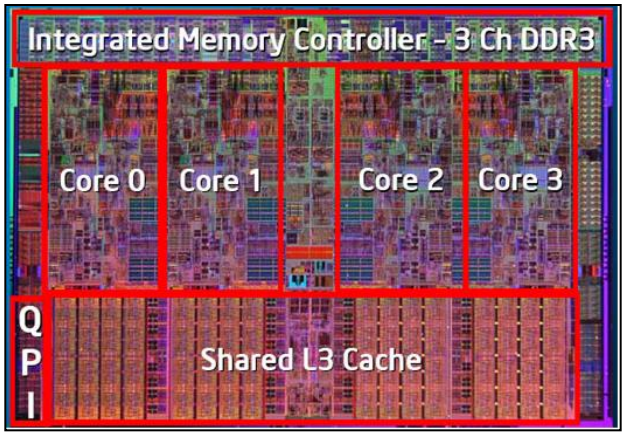
Parallel computers

The vast majority of algorithms in this course are ***serial algorithms*** suitable for running on a uniprocessor computer.

Parallel computers have become increasingly common:

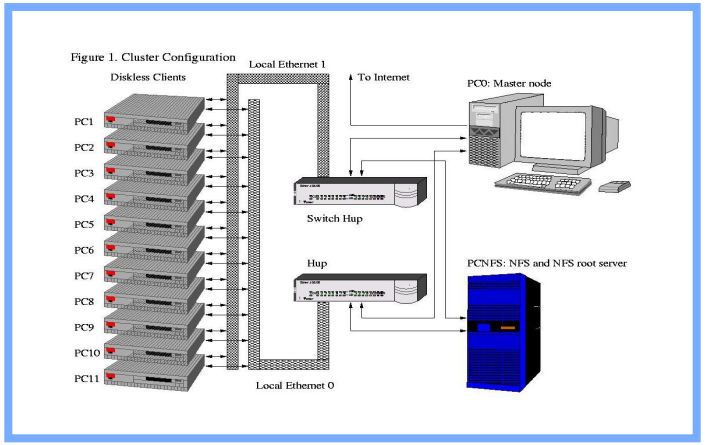
- Relatively inexpensive desktop and laptop ***chip multiprocessors*** contain a single multi-core integrated-circuit chip.
- ***Clusters*** built from individual computer (often simple PC-class machines) with a dedicated network interconnecting them.
- The highest-priced machines are ***supercomputers***.

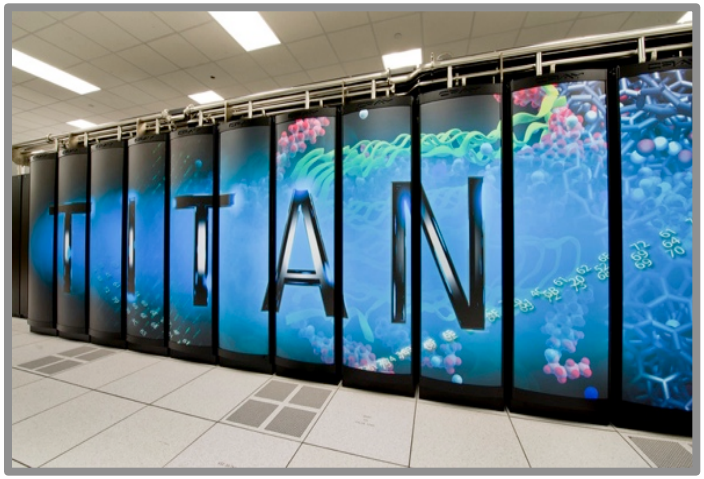
Parallel computers





Chip multiprocessors





Titan Supercomputer



Cluster

Shared vs. distributed memory

No single model for parallel computing has gained as wide acceptance. For example,

- Some parallel computers feature ***shared memory***, where each processor can directly access any location of memory.
- Other parallel computers employ ***distributed memory***, where each processor's memory is private, and an explicit message must be sent between processors.

Every new laptop and desktop machine is now a ***shared-memory*** parallel computer, and the trend appears to be toward shared-memory multiprocessing.

Dynamic multithreaded programming

- Unfortunately, programming a shared-memory parallel computer directly using static threads is ***difficult*** and ***error-prone***.
- ***Concurrency platforms*** provide a layer of software that coordinates, schedules, and manages the parallel-computing resources.
- One important class of concurrency platform is ***dynamic multithreading***, which allows programmers to specify parallelism in applications without worrying about other issues.

Fibonacci numbers

Recursive definition:

$$F_n = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2; \end{cases}$$

0 1 1 2 3 5 8 13 21 34 $\cdots$

Naive recursive algorithm: $\Omega(\phi^n)$ (exponential time), where $\phi = (1+\sqrt{5})/2$ is the ***golden ratio***.

Dynamic multithreading Fibonacci

```
FIB(n)
1   if n ≤ 1
2       return n
3   else x = FIB(n − 1)
4        y = FIB(n − 2)
5   return x + y
```

Multithread version

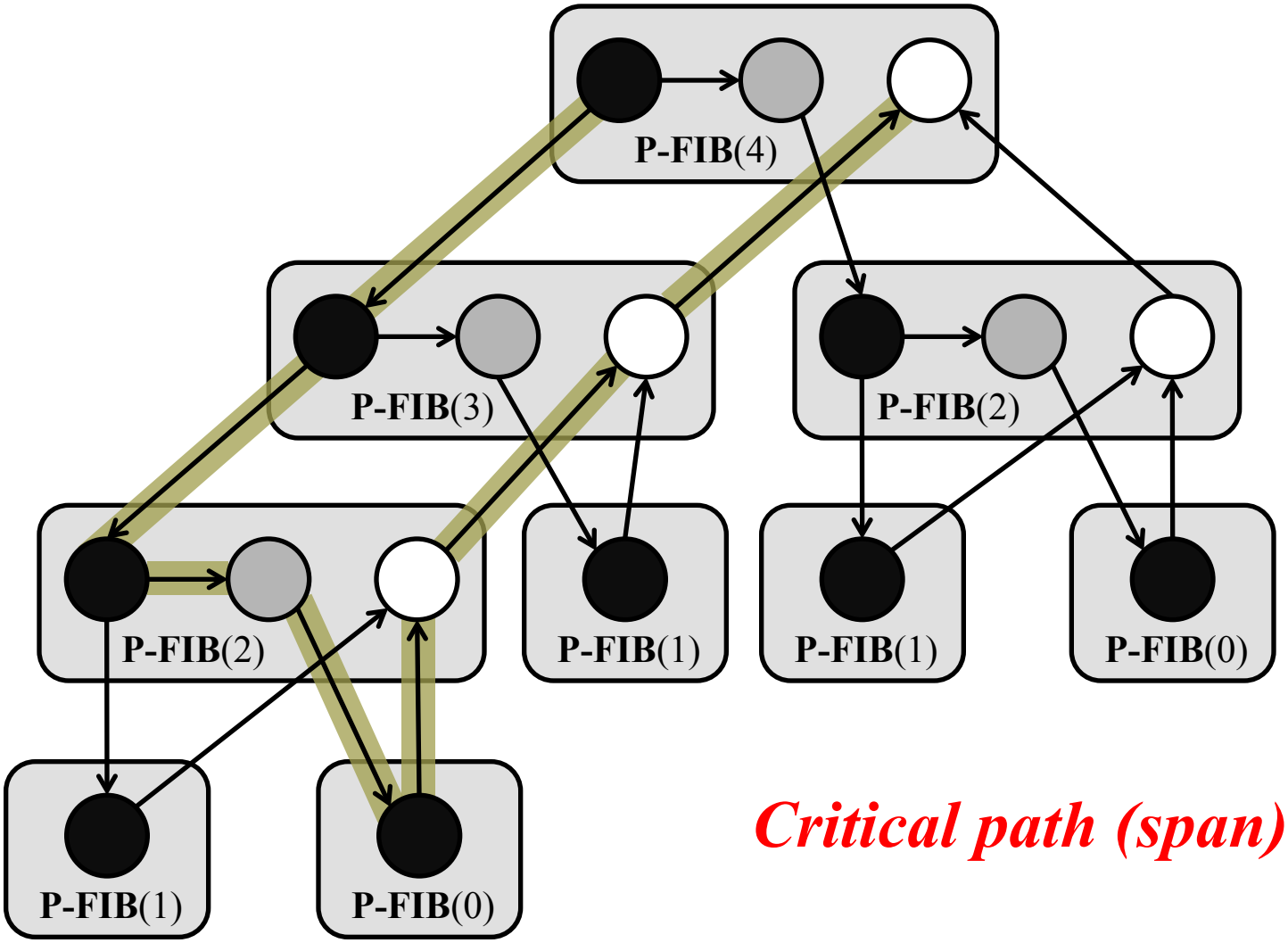
```
P-FIB(n)
1   if n ≤ 1
2       return n
3   else x = spawn P-FIB(n − 1)
4        y = P-FIB(n − 2)
5        sync
6        return x + y
```

A model for multithreaded execution

We can think of a ***multithreaded computation*** as a directed acyclic graph $G = (V, E)$, called a ***computation dag***.

- The vertices in V are instructions, and
- The edges in E represent dependencies between instructions, where $(u, v) \in E$ means that instruction u must execute before instruction v.

A computation dag for P-FIB(4)



Critical path (span)

Performance measures

We can gauge the theoretical efficiency of a multi-threaded algorithm by using two metrics:

- The ***work*** of a multithreaded computation is the total time to execute the entire computation on one processor.
- The ***span*** is the longest time to execute the strands along any path in the dag.

Notation

The actual running time of a multithreaded computation depends not only on its word and its span, but also on how many processors are available and how the scheduler allocates strands to processors.

T_P: the running time of an algorithm on P processors.

T_1: the running time of an algorithm on a single processor.

T_∞: the running time of an algorithm on an unlimited number of processors.

Notation (continue)

T_1/T_p: the ***speedup*** of a computation on P processors. when $T_1/T_p = \Theta(P)$, the computation exhibits ***linear speedup***, and when $T_1/T_p = P$, we have ***perfect linear speedup***.

T_1/T_∞: the ratio of the work to the span gives the ***parallelism*** of the multithreaded computation. As an ***upper bound***, the parallelism gives the maximum possible speedup that can be achieved.

$(T_1/T_\infty)/P$: the ***(parallel) slackness*** of a multithreaded computation executed on an ideal parallel computer with P processors.

Greedy schedulers

Theorem. On an ideal parallel computer with P processors, a greedy scheduler executes a multithreaded computation with work T_1 and span T_∞ in time

$$T_P \le T_1 / P + T_\infty$$

Corollary. The running time T_P of any multithreaded computation scheduled by a greedy scheduler on an ideal parallel computer with P processors is within a factor of 2 of optimal.

$$\begin{aligned} T_P &\le T_1 / P + T_\infty \\ &\le 2 \cdot \max(T_1 / P, T_\infty) \\ &\le 2T_P^* \end{aligned}$$

Analyzing multithreaded P-FIB(n)

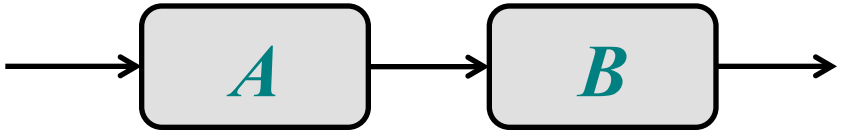
$$T_1(n) = T(n) = \Theta(\phi^n)$$

$$\begin{aligned} T_\infty(n) &= \max(T_\infty(n-1), T_\infty(n-2)) + \Theta(1) \\ &= T_\infty(n-1) + \Theta(1) \\ &= \Theta(n) \end{aligned}$$

The parallelism of **P-FIB**(n) is $T_1(n) / T_\infty(n) = \Theta(\phi^n / n)$, which grows dramatically as n gets large. Thus, on even the largest parallel computers, a modest value for n suffices to achieve near perfect liner speedup for **P-FIB**(n).

Composed subcomputations

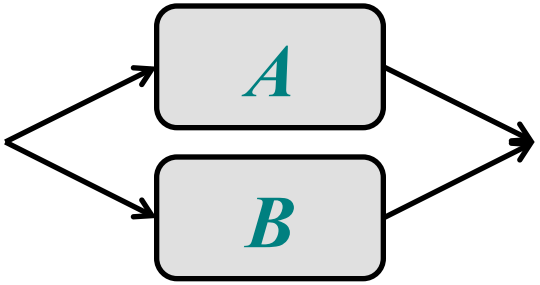
- Two subcomputations are joined in ***series***.



Work: $T_1(A \cup B) = T_1(A) + T_1(B)$

Span: $T_\infty(A \cup B) = T_\infty(A) + T_\infty(B)$

- Two subcomputations are joined in ***parallel***.



Work: $T_1(A \cup B) = T_1(A) + T_1(B)$

Span: $T_\infty(A \cup B) = \max(T_\infty(A), T_\infty(B))$

Parallel loops

As an example, consider the problem of multiplying an $n \times n$ matrix $A = (a_{ij})$ by an n-vector $x = (x_j)$. The resulting n-vector $y = (y_i)$ is given by the equation $y_i = \sum_{j=1}^{n} a_{ij} x_j$

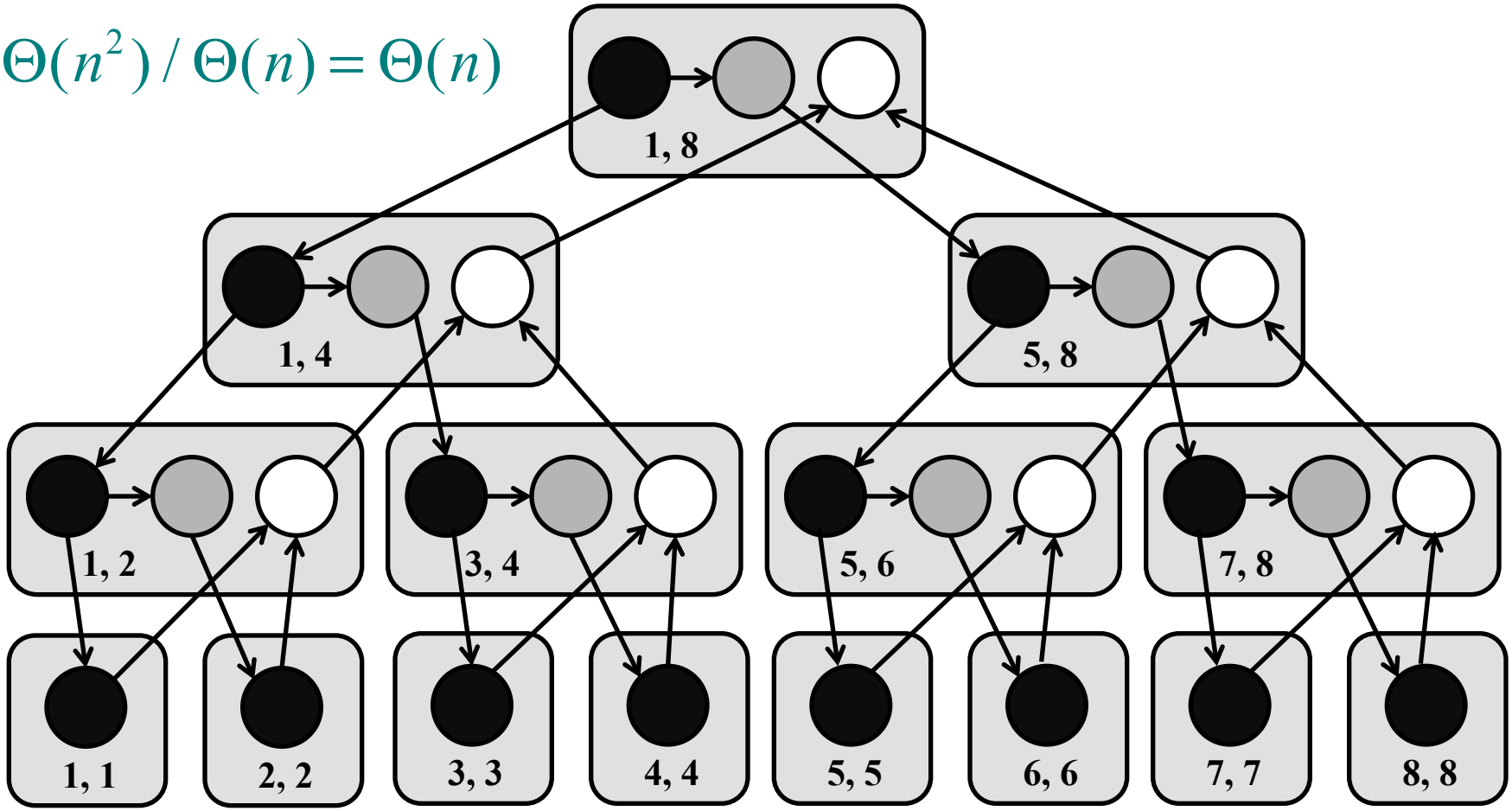
```
MAT-VEC(A, x)
1    n = A.rows
2    let y be a new vector of length n
3    parallel for i = 1 to n
4        y_i = 0
5    parallel for i = 1 to n
6        for j = 1 to n
7            y_i = y_i + a_ij x_j
8    return y
```

Divide-and-conquer subroutine

```
MAT-VEC-MAIN-LOOP(A, x, y, n, i, k)
1    if i = = k
2        for j = 1 to n
3            y_i = y_i + a_ij x_j
4    else mid = (i + k) / 2
5        spawn MAT-VEC-MAIN-LOOP(A, x, y, n, i, mid)
6        MAT-VEC-MAIN-LOOP(A, x, y, n, mid + 1, k)
7        sync
```

Dag representation

$\Theta(n^2) / \Theta(n) = \Theta(n)$



$T_1(n) = \Theta(n^2)$

$T_\infty(n) = \Theta(\lg n) + \max_{1 \le i \le n} iter_\infty(i) = \Theta(n)$

Race conditions

Race conditions are the bane of concurrency. Famous race bugs include:

- The Therac-25 radiation therapy machine, which killed three people and injured several others.
- North American Blackout of 2003, which left over 50 million people without power.

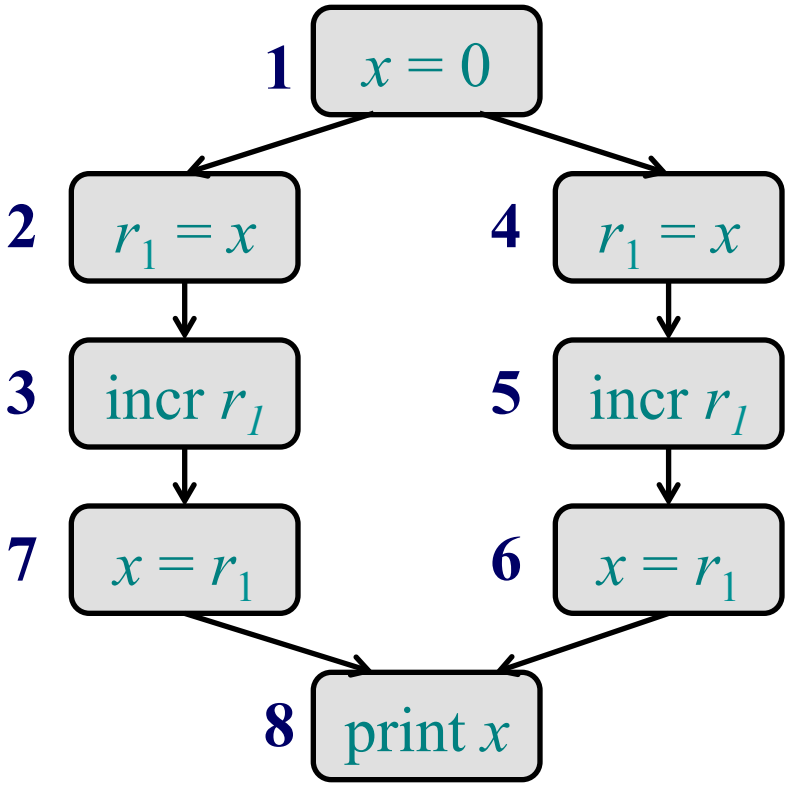
These pernicious ***bugs*** are notoriously ***hard*** to ***find***. You can run tests in the lab for days without a failure only to discover that your software sporadically crashes in the field.

Be careful!

Example of race conditions

A ***determinacy race*** occurs when two logically parallel instructions access the ***same memory location*** and at least one of the instructions performs a ***write***.

```
RACE-EXAMPLE( )
1    x = 0
2    parallel for i = 1 to 2
3        x = x + 1
4    print x
```



Matrix-vector multiplication

MAT-VEC(A, x)

```
1    n = A.rows
2    let y be a new vector of length n
3    parallel for i = 1 to n
4        y_i = 0
5    parallel for i = 1 to n
6        parallel for j = 1 to n
7            y_i = y_i + a_ij x_j
8    return y
```

Is it correct?

A chess lesson

This is a ***true story*** that occurred during the development of the world-class multithreaded chess-playing program. The program was prototyped on a 32-processor computer but was ultimately to run on a supercomputer with 512 processors.

At one point, the developers incorporate an optimization into the program that reduced its running time on an important benchmark on the 32-processor machine from $T_{32} = 65$ seconds to $T'_{32} = 40$ seconds.

The developers used the ***work*** and ***span*** performance measures to conclude that the optimized version, which was faster on 32 processors, would actually be slower than the original version on 512 processors. As a result, they ***abandoned*** the "***optimization***".

What is happened?

Original version:

$T_1 = 2048$ and $T_\infty = 1$.

We use $T_P = T_1 / P + T_\infty$ as an approximation to the running time on P processors.

$T_{32} = 2048/32 + 1 = 65$ $\quad$ $T_{512} = 2048/512 + 1 = 5$

Optimized version:

$T_1' = 1024$ and $T_\infty' = 8$.

$T_{32} = 1024/32 + 8 = 40$ $\quad$ $T_{512} = 1024/512 + 8 = 10$

The optimization that sped up the program on 32 processors would have made the program ***twice*** as ***slow*** on 512 processors!

Multithreaded matrix multiplication

P-SQUARE-MATRIX-MULTIPLY(A, B)

```
1   n = A.rows
2   let C be a new n × n matrix
3   parallel for i = 1 to n
4       parallel for j = 1 to n
5           c_ij = 0
6           for k = 1 to n
7               c_ij = c_ij + a_ik b_kj
8   return C
```

$T_1(n) = \Theta(n^3)$ $\quad T_\infty(n) = \Theta(\lg n) + \Theta(\lg n) + \Theta(n) = \Theta(n)$

The parallelism is $T_1(n) / T_\infty(n) = \Theta(n^3) / \Theta(n) = \Theta(n^2)$

Divide-and-conquer algorithm

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

Then, we can write the matrix product as

$$C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$= \begin{pmatrix} A_{11}B_{11} & A_{11}B_{12} \\ A_{21}B_{11} & A_{21}B_{12} \end{pmatrix} + \begin{pmatrix} A_{12}B_{21} & A_{12}B_{22} \\ A_{22}B_{21} & A_{22}B_{22} \end{pmatrix}$$

Divide-and-conquer parallel algorithm

```
P-SQUARE-MULTIPLY-RECURSIVE(C, A, B)
1     n = A.rows
2     if n = = 1
3        c_11 = a_11 b_11
4     else let T be a new n × n matrix
5           partition A, B, C, and T into n/2 × n/2 submatrices.
6           spawn P-MATRIX-MULTIPLE-RECURSIVE(C_11, A_11, B_11)
7           spawn P-MATRIX-MULTIPLE-RECURSIVE(C_12, A_11, B_12)
8           spawn P-MATRIX-MULTIPLE-RECURSIVE(C_21, A_21, B_11)
9           spawn P-MATRIX-MULTIPLE-RECURSIVE(C_22, A_21, B_12)
10          spawn P-MATRIX-MULTIPLE-RECURSIVE(T_11, A_12, B_21)
11          spawn P-MATRIX-MULTIPLE-RECURSIVE(T_12, A_12, B_22)
12          spawn P-MATRIX-MULTIPLE-RECURSIVE(T_21, A_22, B_21)
13          P-MATRIX-MULTIPLE-RECURSIVE(T_22, A_22, B_22)
14          sync
15          parallel for i = 1 to n
16                parallel for j = 1 to n
17                      c_ij = c_ij + t_ij
```

Computational complexity analysis

$$T_1(n) = 8T_1(n/2) + \Theta(n^2)$$
$$= \Theta(n^3)$$

$$T_\infty(n) = T_\infty(n/2) + \Theta(\lg n)$$
$$= \Theta(\lg^2 n)$$

Its parallelism as $T_1(n)/T_\infty(n) = \Theta(n^3/\lg^2 n)$, which is very high.

How about multithreading Strassen's method?

Multithreaded merge sort

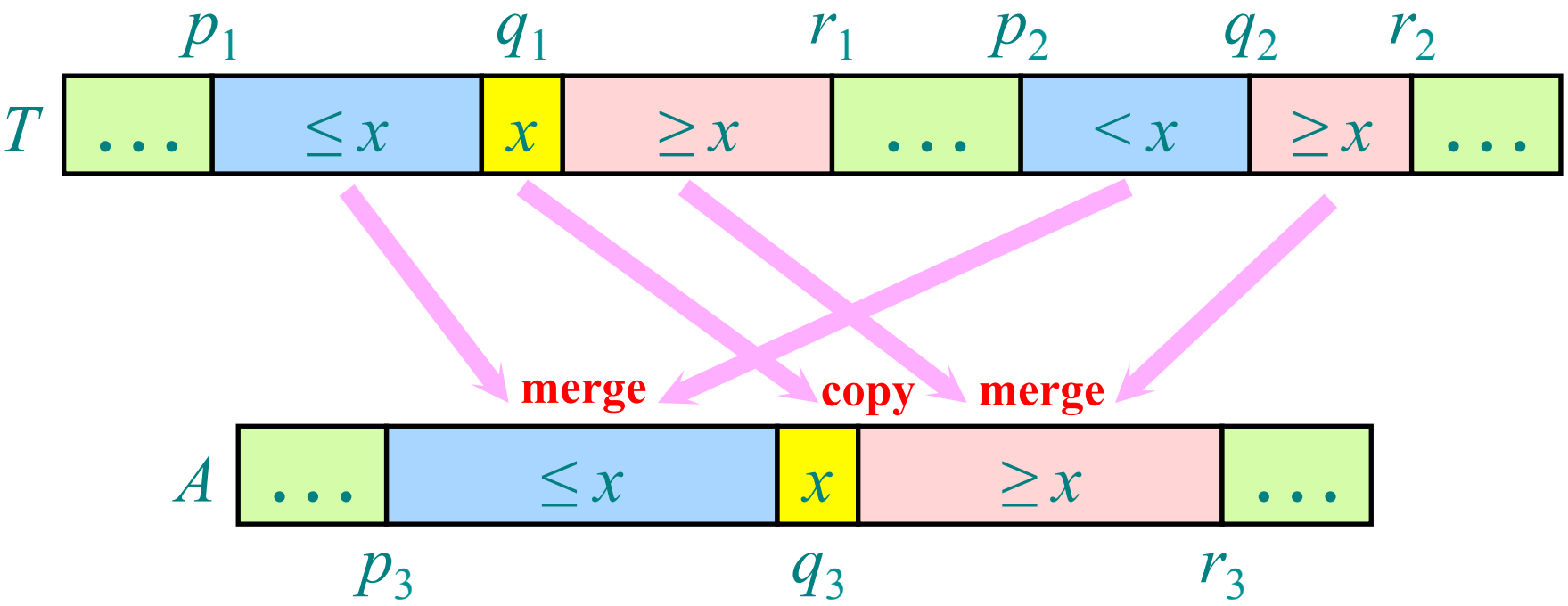
```
P-MERGE-SORT(A, p, r)
1    if p < r
2        q = (p + r) / 2
3        spawn P-MERGE-SORT(A, p, q)
4        P-MERGE-SORT(A, q + 1, r)
5        sync
6        MERGE(A, p, q, r)
```

$$T_1(n) = 2T_1(n/2) + \Theta(n) = \Theta(n \lg n)$$

$$T_\infty(n) = T_\infty(n/2) + \Theta(n) = \Theta(n)$$

$$T_1(n) / T_\infty(n) = \Theta(\lg n)$$

Multithreaded merging of subarrays



Multithreaded merging algorithm

```
P-MERGE(T, p1, r1, p2, r2, A, p3)
1     n1 = r1 − p1 + 1
2     n2 = r2 − p2 + 1
3     if  n1 < n2    // ensure that n2 ≥ n1
4     exchange p1 with p2
5     exchange r1 with r2
6     exchange n1 with n2
7     if  n1 = = 0
8         return
9     else  q1 = (p1 + r1) / 2
10            q2 = BINARY-SEARCH(T[q1], T, p2, r2)
11            q3 = p3 + (q1 − p1) + (q2 − p2)
12            A[q3] = T[q1]
13            spawn P-MERGE(T, p1, q1 − 1, p2, q2 − 1, A, p3)
14            P-MERGE(T, q1 + 1, r1, q2, r2, A, p3 + 1)
15            sync
```

Analysis of multithreaded merging

The key is to understand that in the worst case, the maximum number of elements in either of the recursive calls can be at most $3n/4$.

Because lines 3-6 ensure that $n_2 \le n_1$, it follows that $n_2 = 2n_2/2 \le (n_1 + n_2)/2 = n/2$.

In the worst case, one of the two recursive calls merges $\lfloor n_1/2 \rfloor$ elements of $T[p_1 .. r_1]$ with all n_2 elements of $T[p_2 .. r_2]$, and hence the number of elements involved in the call is

$$\begin{aligned} \lfloor n_1/2 \rfloor + n_2 &\le n_1/2 + n_2/2 + n_2/2 \\ &= (n_1 + n_2)/2 + n_2/2 \\ &\le n/2 + n/4 = 3n/4 \end{aligned}$$

Analysis of multithreaded merging

We obtain the following recurrence for the worst-case span of multithreaded merging

$$PM_\infty(n) = PM_\infty(3n/4) + \Theta(\lg n)$$
$$= \Theta(\lg^2 n)$$

The recurrence of multithreaded merge sort

$$T_\infty(n) = T_\infty(n/2) + \Theta(\lg^2 n)$$
$$= \Theta(\lg^3 n)$$

The parallelism is

$$T_1(n)/T_\infty(n) = \Theta(n \lg n)/\Theta(\lg^2 n)$$
$$= \Theta(n/\lg^2 n)$$

which is much better both in theory and in practice.

Any question?

Xiaoqing Zheng
Fundan University